

TRD — City Life: Lonavala

Rules for the coding agent (save credits)

- Do exactly what each task says. No extra features, no refactors, no tests, no docs unless asked.
- Use ONLY the libraries listed below. Do not install anything else.
- Keep code simple; JavaScript or TypeScript, minimal files. Do not rewrite files that already work.
- Do not run long browser/test loops. One `npm run build` check at the end of each task is enough.

Stack

- Next.js (App Router) + Tailwind CSS
- Map: `leaflet` + `react-leaflet` (OpenStreetMap tiles, no key)
- DB: Supabase Postgres via `@supabase/supabase-js`
- LLM: Gemini via `@google/generative-ai` (use a free Flash model, e.g. `gemini-2.5-flash`)
- Weather: Open-Meteo REST (no key)
- Routing: OSRM public server `https://router.project-osrm.org` (no key)
- Deploy: Vercel (from GitHub)

Env vars (.env.local and Vercel)

```
GEMINI_API_KEY=  
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=
```

Folder structure

```
app/  
  page.tsx           # Explore map + filters + weather banner  
  compare/page.tsx   # Best vs worst + side-by-side  
  report/page.tsx    # Citizen report form  
  assistant/page.tsx # AI chat  
  api/reports/route.ts # POST (classify with Gemini + insert), GET (list)
```

```

  api/chat/route.ts      # POST chat answered from places.json +
reports
components/             # Map.tsx (client-only, dynamic import
ssr:false), PlaceCard.tsx, Nav.tsx
data/places.json        # seed places (provided)
lib/score.ts            # overall score
lib/geo.ts              # haversine + route risk check
lib/supabase.ts

```

Database (run in Supabase SQL editor)

```

create table reports (
  id uuid primary key default gen_random_uuid(),
  created_at timestampz default now(),
  text text not null,
  lat double precision not null,
  lng double precision not null,
  photo_url text,
  category text,      -- safety | cleanliness | traffic | weather |
other
  severity int,      -- 1-5
  summary text
);
alter table reports enable row level security;
create policy "public read" on reports for select using (true);
create policy "public insert" on reports for insert with check (true);

```

Photo: for the MVP store a small base64 data URL (resize to <200KB client-side) or skip the photo. No storage bucket needed.

Data model (data/places.json item)

```

id, name, category, lat, lng, price_level (0 free - 4 luxury),
approx_cost_inr,
scores{safety,cleanliness,affordability,rating,accessibility}, tags[],
description, monsoon_restricted(bool), note

```

Score (lib/score.ts)

```

overall = 0.35*safety + 0.20*cleanliness + 0.20*affordability +
0.15*rating + 0.10*accessibility

```

Weights come from sliders in the UI (default above). Best = top 3, Worst = bottom 3 within a category filter.

Weather (Open-Meteo, Lonavala 18.7546, 73.4062)

GET `https://api.open-meteo.com/v1/forecast?latitude=18.7546&longitude=73.4062¤t=temperature_2m,precipitation,weather_c`
Banner rule: if current precipitation > 2 mm OR max rain probability next 6h > 60% -> show "Heavy rain alert: avoid waterfalls, dams and viewpoints" and mark `monsoon_restricted` places with a red badge.

Safer route (`lib/geo.ts`)

- Call `https://router.project-osrm.org/route/v1/driving/{lng1},{lat1};{lng2},{lat2}?overview=full&geometries=geojson&alternatives=true`
- For each route, count risk points (places with category `risk` or `monsoon_restricted`) within 400 m of any route point (haversine, sample every ~10th point).
- Show the route with the fewest risk points in green; list nearby risks as warnings. If all routes pass risks, show the warning anyway.

AI endpoints

POST /api/reports — body `{text, lat, lng, photo?}` .

Gemini prompt (JSON-only output): "Classify this citizen report from Lonavala. Return ONLY JSON: {category: one of safety|cleanliness|traffic|weather|other, severity: 1-5, summary: max 12 words}. Report: <text>". Parse safely (strip code fences); on failure use `{category:'other', severity:2, summary:text.slice(0,80)}` . Insert into Supabase, return the row.

POST /api/chat — body `{message}` .

Build a prompt with a compact list of places (name, category, price, scores, notes) plus the 10 latest reports. System text: "You are a Lonavala city guide. Answer ONLY from the data given. Be brief, mention safety warnings, give 2-3 suggestions with reasons. If the data lacks the answer, say so." Return `{answer}` .

Non-functional

- Mobile-friendly layout; map height ~60vh on phones.
- Leaflet must be loaded client-side only (`dynamic(() => import(...), { ssr: false })`); fix default marker icons or use colored circle markers by category.
- API keys only in server routes (never `NEXT_PUBLIC_` for Gemini).

Deployment

1. `git init` , commit, push to GitHub (`city-life-lonavala`).

2. Vercel -> Import repo -> add the 3 env vars -> Deploy.
3. Test live URL on phone; add the live link to README.